Empirical Experiment of Algorithms for ITC 2007 Examination Timetabling

Hoang Le*†
Tennessee Tech. University
hhh1e42@tntech.edu

Ian Cornin*†
Tennessee Tech. University
imcornin42@tntech.edu

Abstract

This study was conducted solely to understand how different algorithms perform on the ITC 2007 Examination Timetabling Problem [9]. The method spans four families: construction, trajectory, population, and exact/hybrid. They were originally implemented in Python, but due to runtime limitations, it was decided that handling the thirteen different algorithms in C++ 20, sharing a delta-evaluation pipeline and data structure, was a better choice for research. OR-Tools CP-SAT [11] is invoked through the same harness as the heuristics with the intention to remove variance from the comparison so that dynamics are the sole axis of variation. The ITC 2007 data access was limited to only eight publicly available exams (242–1,096 exams), but we also included our synthetic scaling ladder (50–1,000 exams) in a separate experimental phase for additional testing, as well as a one-dimensional parameter sweep. Each configuration is run on a different number of seeds, which are documented on our GitHub. The results establish the following: (i) Trajectory metaheuristics occupy the quality vs runtime Pareto frontier with GVNS, Tabu, and Kempe-chain holding the top ranking positions 1.88, 2.38, 3.50; (ii) population methods form a distinct secondary tier, led by ABC position 3.75; (iii) CP-SAT returned a proof on very ITC 2007 but is bounded below 900 exams and 300-second budget due to computation availability; and (iv) a warm-started chain ALNS → Kempe → Tabu, discovered by the chain technique, outperforms every single method configuration at a norm geometric mean score of 0.9282.

* I, Hoang Le, conducted the following with GenAI (Claude and ChatGPT) assistant: brainstorming, research, review, implementation of all code, experimental design, plotting results, data analysis, report/slides writing, and delivery of this paper. Ian Cornin initially stepped in to develop the roadmap for the dataset plan and risk assessment (which I later refactored), but due to external workload, Ian could not contribute much to this research project. Communication was established between the two of us, but Ian did not follow up much past the planning phase. So, moving forward in this research report, I'll be the primary researcher.

† Work performed for CSC2400 at Tennessee Tech. University

‡ April 2026 Code and data: *https://github.com/Dialovos/exam-scheduling-algos-analyze*

Initial Research Questions

Four research questions were proposed during the initial planning stage before the experiment was conducted. The four are descriptive rather than theoretical, which is consistent with the empirical analysis. The following questions were:

Q1. Which Algorithms produce the best quality within the time limit on ITC 2007 (for simplicity, it was later refactored to be in a family group)?

Q2. Which algorithms are Pareto-optimal in runtime and soft-penalty plan under a practical compute budget?

Q3. Are the winning algorithms ranking dependent on aggressive tuning, or does hyperparameter tuning not matter as much?

Q4. At what instance size does CP-SAT stop returning proof within a 300-second budget?

Of the four questions asked, not enough time was allocated to answering Q4. This was partly due to how the integer programming (IP) was implemented. Firstly, it terminated once an optimal solution was found, rather than backtracking over every possible path to find a better one, and, as a result, it was faster than the trajectory that had to run its full course. Secondly, many of the search patterns were restricted to avoid extra search. Finally, the initial implementation of IP could run up to 150 exams before struggling to finish past that threshold within a timely budget. Given the specified period, answering Q4 was ruled out as biased toward brute-force algorithms, but my intuition suggests it would fall within the 200 range.

1. Problem Premise

The capacitated examination timetabling problem assigns a set of exams E to (period, room) pairs subject to hard and soft constraints.

- Exams E – duration and enrollment
- Periods P – duration limit, day, and optional penalties
- Rooms R – capacity limit and optional penalties
- Students S – exam subsets, inducing a conflict graph on E
- Period hard constraints
- Room hard constraints
- And soft constraints weighting.

A feasible assignment π: E → P * R respects every hard constraint and minimizes the weighted total of soft constraints. The feasibility subproblem reduces graph coloring [13], and the full problem is NP-hard. Following the rule of ITC 2007 requirement [9], feasibility dominates over soft for rule 7 (d, s) ordinal pair, but I took a different approach while still attempting to preserve it:

$$f(\pi) = 100{,}000 * h(\pi) + s(\pi)$$

Where $h(\pi)$ is the count for hard-constraint violation and $s(\pi)$ is the weighted soft penalty sum. The five orders of magnitude ensure that no infeasible solution dominates the feasible one. $h(\pi)$ aggregates violation for: student double booking, room capacity, period-duration, period hard-constraint (coincidence, exclusion, after), and room-exclusive, while $s(\pi)$ aggregates soft weightings for: two-in-a-row, two-in-a-day, period spread, non-mixed durations, front-load, period penalty, and room penalty.

Datasets ITC 2007

Set	Number of Exams
Set1	607
Set2	870
Set3	937
Set4	273
Set5	1018
Set6	242
Set7	1096
Set8	598

Synthetic ladder instances ranging from $n = 50$ to $n = 1{,}000$ is used for the scaling analysis in §5. Set one was used as the main benchmark; set two fell below the CP-SAT cliff, set three was above the CP-SAT cliff, and set eight was anomalous and excluded from CP-SAT. All synthetic instances use the competition present at seed 42 in GitHub‡. See §4 for rules that were skipped to save time on the result.

2. Algorithms Explanations

2.1 Construction Family

DSatur Greedy is a constructive saturation-degree heuristic (building a new schedule from scratch) that other algorithms mentioned later will rely on to improve the known solution (though it is unlikely to produce a feasible solution). The main rule is that at every step, it looks for the exams that are currently the hardest to place by finding the unscheduled exam that conflicts most with the existing ones in the current calendar. Handling the hardest schedule first avoids being backed into a corner later. If two exams are intended to at equally difficult, the focus shifts to student headcount, and the exams with the largest headcounts will be scheduled first. DSautr acts as a global warm start and isn't meant to find a perfect or optimal solution. It laid the foundation, giving other algorithms a head start to do the heavy lifting later on.

2.2 Trajectory Family

Tabu Search [3] at its core is an optimization tool that explores different solutions by always taking the best next step; however, it uses a memory system to avoid getting stuck in a loop. Whenever it makes a move, the inverse of the most recent move is forbidden for a set number of turns, promoting exploration of new solutions

rather than wasting time backtracking. There is an exception to the rule mentioned above: if a forbidden move would create a new improvement to the best-known fitness, the algorithms will ignore the rules to make that move.

Simulated Annealing (SA, Multi-Neighborhood) [8] is inspired by how metals cool and harden. In the initial state, when the temperature is high, it will occasionally accept worse solutions (uphill moves) to avoid getting stuck in a dead end. As time passes during its cooling stage, it becomes stricter and accepts only moves that actively improve the solution. To find better solutions, it alternates between different ways of handling the current schedule: (i) moving just one item to a new spot (single-move); (ii) trading the place of two items (swap); (iii) swapping the entire linked sequence of items all at once (Kempe-chain). The starting "heat," or in other words, how the algorithms are to accept bad moves at the beginning, is calibrated for every different exam.

The Kempe-Chained Search safely swaps an entire group of exams at once to shake things up. The two-coloring method looks at just two specific time slots and finds a chain of conflicting exams that alternate between these two time slots, then it flips the whole chain simultaneously. Because of the swapping of the whole alternating chain at the same time, none of those specific exams will conflict with each other. Doing a sequential move for each exam can cause it to get stuck, which is why moving in chunks (an atomic chain move) lets the algorithms make a massive leap to a different layout, helping discover new solutions faster.

Great Deluge (GD) [2] is like hiking in a valley to find the lowest point, but it's raining, and the rainwater sets a strict ceiling on how bad (high up) the solution can be. The algorithm handles this by adjusting the schedule, accepting the worst move as long as the new schedule remains below that water level. The way to prevent it from getting stuck in a shallow valley is to let it climb over a small hill into a deeper one. The level-decay rate (flood) as time passes, the water level progressively drops (gets stricter), shrinking the acceptable pool of bad moves and slowly squeezing the algorithm into better solutions until it gets trapped at the lowest point.

Late Acceptance Hill Climbing (LAHC) [1] addresses the problem that standard hill climbing often gets stuck because it refuses to accept a move that slightly worsens the situation by changing what it compares the new move against. The method is late comparison, or in other words, the algorithm keeps a running memory of its scores from the last L turns (e.g., the last 50 steps). When deciding whether to keep the new move, it doesn't compare it to the schedule from a moment ago; instead, it compares it to the scores from exactly 50 steps ago. If the new move is better than the older score, it accepts the move even if it is temporarily worse than the previous step. This allows the algorithm to walk downhill for a little while, as long as it's doing better than before (i.e., not stuck at a dead end). Unlike SA or Tabu, which require multiple rules, LAHC is easy to set up with a single parameter: the length of the memory list L.

General Variable Neighborhood Search (GVNS) [5] uses the whole toolbox of different moves and organizes them sequentially from the smallest to the largest change. The idea is to always perfect the schedule by using its smallest, most precise tools first, and make tiny improvements until it gets stuck at a dead end (a local optimum). To escape once it gets stuck, switch to a larger neighborhood tool to make a broader change and shake up the exam out of the dead end. This will land the algorithm in a new, unexplored area after the big shake, and it will return to using its smaller tool there. This cycle repeats in the hope of achieving that perfect schedule like an overachiever.

Adaptive Large Neighborhood Search (ALNS) [12] learns the best way to the solution by iteratively breaking down a solution and reassembling it through destruction and repair. Instead of making tiny changes to the solution, ALNS makes large changes to escape the local suboptimal trap by removing a chunk of the current solution (destroy operator) and taking those unassigned elements and putting them back into the solution in a hopefully more efficient way (repair) known as the *destroy and repair* method. It doesn't have a single way to perform the destroy-and-repair method, which comes in handy since it's adaptive. The algorithm keeps score, and if a specific combination of a destroy and repair tool yields a massive improvement to the schedule, it will learn from this success and increase the weight of using that specific tool in the future. It plays out the number of turns (segment),

looks back at how well it performed during that chunk of time, and updates its preference all at once before the next segment.

2.3 Population Family

Memetic Genetic Algorithm (GA) [4] operates like biological evolution by taking a population of different schedules, breeding the best ones together to produce offspring, and repeating the process. In the memetic part, unlike in pure genetics, the offspring also get a chance to learn and improve themselves individually. When combining two parents' schedules to make a child, it carefully splices them together while preserving the relative sequence of the exams that seemed to work well for the parents. As soon as a new child schedule is created, it isn't just tossed into the survival pool; the algorithms spend a little time doing a quick fine-tuning (local search) on the specific schedule to polish it up and maximize its potential. The absolute best schedules are granted immunity (elitism), ensuring they survive for the next generation without being accidentally destroyed. It protects the legal schedule, rather than the broken but promising one.

Artificial Bee Colony (ABC) [7] mimics how a swarm of bees works together to find the best patches of flowers (optimal schedule) by dividing the workforce into three roles: employed bees, onlooker bees, and scout bees. (i) The employed bees are assigned to a food source (current schedule) and explore the immediate neighborhood around the schedule, making small changes to see if they can find something slightly better nearby. (ii) The onlooker bee needs to wait in their hive and watch the employed bee report their findings; the better the schedule (probability proportional to fitness), the more likely an onlooker bee will be convinced to fly out and help explore that area. This allows for the computation budget to be focused on a promising solution. (iii) Lastly, if the employed bee has changed the same schedule for too long without finding any improvement (exhausted food), it abandons the schedule and turns into a scout bee jumping to a random schedule to explore.

Whale Optimization Algorithm (WOA) [10] is inspired by coordinated humpback whale hunting in groups that trap prey near the surface. The strategies are to hunt for the best schedule using two move-based on whale behavior: encircling and bubble-net. (i) The whale identifies the best schedule found so far and slowly shrinks its search radius around it, closing the gap on the target, known as encircling. (ii) They also search in a tightening, upward spiral pattern and constantly flip a coin to decide whether to use the shirking circle or spiral to close in on the best solution schedule. They swim in a continuous space; however, an exam schedule is strictly discrete. To fix this, the algorithm calculates where the whale wants to swim in the continuous space and then snaps or rounds that position to the closest legal exam schedule.

Harris Hawks Optimization, Hybridized (HHO+) [6] is based on a coordinated team of hunting Harris Hawks trying to exhaust and trap the rabbit. Like the encircling strategy of WOA, the hawk close in on the best schedule found so far and shrinks its search radius around the target to narrow the gap toward a better solution. While aggressively searching for a specific area, it uses a Levy flight, a mix of many tiny steps combined with occasional large leaps. The hybrid part means that once the hack has finished their main hunt and trapped a promising schedule, it'll switch its approach to perform a final round of local search to polish that specific schedule to the best of its ability.

2.4 Exact / Hybrid Family

OR-Tool CP-SAT (IP) [11] is a solver (developed by Google) to throw a kitchen sink of advanced mathematical tricks at a problem to find the best schedule. The trick (engine) generates rules (lazily) only at the exact moment the algorithm needs them, instead of loading millions of rules into its memory at once. It stops to

analyze what failed every time it hits a dead end, so it never makes that exact combination of mistakes again (conflict-driven clause learning). Next are LP relaxation and cutting planes, which temporarily pretend that a fraction of an exam is legal, just so it can use incredibly fast math (linear programming) to estimate where the best answers are, then cut away the bad math to close in on the real schedule. The algorithm detects when two schedules are essentially identical and prevents itself from waiting time by checking for duplicates through symmetry breaking. Next is the two setup configurations that I've set up: (i) cold starts come scratch with zero hint and give a strict 60 second time limit and run on C++ to brute force a usable solution; then (ii) it handle a solid first draft schedule (seeded by a Tabu solution) as hint and given a longer time limit up to 300 second to utilizes all of the computer processor core at once to fine tune that draft into the best solution.

2.5 Chain-Finder

The chain finder system (tournament search) was built to find the best candidate sequence (chain) by combining multiple algorithms in a pipeline. A tournament search over a warm-start algorithm chain, and a chain is a sequence $[(a_1, p_1), (a_2, p_2), \dots, (a_k, p_k)]$ where each step runs the algorithm a_i with parameter p_i initialized from the previous step's best solution. The tournament uses successive halving with eta-scheduled rungs, a prefix *.sln* cache that amortizes shared chain beginning to save computing budget, early-stop pruning of partial fitness exceeding 2.5x the dataset baseline, and partial-credit scoring for chains whose last step times out. Adject duplicates were disabled, and the maximum chaining length was set to 10.

3. Complexity

Pre-iteration report for the inner loop only; the wall clock for each algorithm depends on the successful use of math shortcuts enabled by delta-evaluation and an implementation-level engineering substrate. Check GitHub for the iteration count of each algorithm §6.

Algorithm	Complexity	Notes
DSatur Greedy	$\Theta(\lvert E\rvert^2)$	1-shot construction with no inner loop
Tabu	$\Theta(k)$	k = single-move neighborhood size with delta-evaluation.
SA	$\Theta(k)$	Single move per step
Kempe	$\Theta(\lvert C\rvert)$ partial eval	$\lvert C\rvert \approx$ chain size, and *partial_eval* restricted to the chain-affected student.
Great Deluge	$\Theta(k)$	Same as the SA inner loop
LACH	$\Theta(1)$	Single comparison history window
GVNS	$\Theta(k_j)$	k_j = current neighborhood size in the order list
ALNS	$\Theta(k * \log n)$	Destroy and repair with priority-queue updates
GA	$\Theta(P * \lvert E\rvert)$	P = population and crossover + delta-polish pre offspring
ABC	$\Theta(P * k)$	Three bee phase passes per generation
WOA	$\Theta(P * n)$	$n = \lvert E\rvert$; and vector style position update over all exams

HHO+	$\Theta(P * n)$	Levy-flight + encircling + local polish per hawk
CP-SAT	Exponential worst case	Solver internal search and empirical runtime in practice

4. Experiment Design

The solver core and every per-algorithm inner loop are C++20 and compiled with g++ -O3 -std=C++20; a Python 3 harness handles ITC file parsing at the CLI level, batch orchestration, result aggregation, plotting, and the auto-tuner driver. OR-Tool CP-SAT is used through both its Python API and a compiled C++ wrapper. The main match results were produced on a Google Colab Pro CPU High-RAM runtime, and all timing reports are in §5. Eight inputs are from the public ITC 2007 examination-track datasets [9], and synthetic inputs were generated by core/generator.py from the competition preset seed 42 and are used for the scaling ladder. For input size, refer to §1. The main match used 7 seeds per cell, with ~730 runs total; the scaling batch used 3 seeds per cell, ~780 runs total; and the sensitivity batch used 3 seeds per cell, ~270 runs total.

The metrics that were used are: (i) soft penalty, (ii) feasibility, (iii) wall clock runtime – capture inside each algorithm C++ inner loop and reported in the batch JSON, for CP-SAT, it was captured from the solver internal clock, (iv) peak memory proc/<pi>/status VmHWM polling, (v) percent cap to CP-SAT – only where CP-SAT return proof for certain set, (vi) mean rank. They all shared a solver core on a single C++20 code path handling all input parsing, problem representation, constraint evaluation, hard-constraint repair, and solution serialization. Each algorithm invokes these primitives through a uniform interface, and no algorithm uses its own evaluator.

Things that deviated from the ITC 2007 rules were rule 6 (single processor machine) to speed up the research trail; rule 8 (early/late/hidden dataset categories) due to an anomaly §6, CP-SAT was excluded; rule 9 (Benchmark program for time normalization) due to time constraint for this search, it was never ran on the ITC 2007 benchmarking program; rule 10 (time limit clause from Sec. 4 of rules) due to time constraint of this research, I impose a wall clock; rule 12 (submit the best score per instance) I took a more rigorous approach for this research and reported the mean ± std rather than the best score.

5. Results

5.1 Main Comparison

Mean soft penalty average over seven seeds with standard deviation per algorithm and dataset. Algorithms are sorted by mean rank across the eight ITC sets, and full numerical data are in graphs/tables/t1_leaderboard.cvs.

Algorithm	*exam_comp_set1*	*exam_comp_set2*	*exam_comp_set3*	*exam_comp_set4*	*exam_comp_set5*
GVNS	10614 +/- 671	2364 +/- 205	21941 +/- 567	25305 +/- 4473	5424 +/- 314
Tabu	8740 +/- 334	2740 +/- 147	25558 +/- 4537	33163 +/- 5950	5789 +/- 612
Kempe	12383 +/- 307	2211 +/- 186	25189 +/- 1155	23999 +/- 3674	10975 +/- 1072
ABC	13663 +/- 708	2841 +/- 88	23287 +/- 1128	28365 +/- 7614	5995 +/- 649
WOA	13862 +/- 977	3578 +/- 290	30832 +/- 1851	34807 +/- 5865	11086 +/- 449
CP-SAT	14715 +/- 1332	3628 +/- 165	25611 +/- 3226	47735 +/- 5472	6428 +/- 637
LAHC	15267 +/- 1446	3952 +/- 280	33516 +/- 3218	32594 +/- 6837	12893 +/- 1152
SA	16249 +/- 1245	6613 +/- 358	40625 +/- 3931	32225 +/- 6410	29308 +/- 1669

ALNS	11252 +/- 437	3117 +/- 101	159157 +/- 28024	45419 +/- 5049	43657 +/- 28087
GA	27975 +/- 8926	7544 +/- 2260	43496 +/- 6935	38595 +/- 6930	13046 +/- 1184
GD	41973 +/- 14178	130115 +/- 45729	173165 +/- 44356	35701 +/- 5674	330410 +/- 49502
HHO+	54546 +/- 2367	123467 +/- 8631	154656 +/- 25171	41154 +/- 6465	264597 +/- 25048
Greedy	68678 +/- 0	183729 +/- 0	277997 +/- 0	62720 +/- 0	412455 +/- 0
IP	8509 +/- 0	2604 +/- 0	-	28162 +/- 0	-

Algorithm	*exam_comp_set6*	*exam_comp_set7*	*exam_comp_set8*	*Mean Rank*	*Best On*
GVNS	31634 +/- 1689	9357 +/- 347	13970 +/- 285	1.88	2
Tabu	30733 +/- 898	8043 +/- 129	12972 +/- 438	2.38	4
Kempe	31449 +/- 1748	10938 +/- 183	14923 +/- 233	3.5	2
ABC	33081 +/- 1719	9592 +/- 336	14339 +/- 279	3.75	0
WOA	33160 +/- 1624	10386 +/- 271	14332 +/- 550	5.88	0
CP-SAT	35336 +/- 3610	10137 +/- 288	14084 +/- 295	6.25	0
LAHC	33057 +/- 2379	12979 +/- 243	15202 +/- 343	6.62	0
SA	33144 +/- 1495	21832 +/- 490	19954 +/- 689	8	0
ALNS	36662 +/- 4518	16186 +/- 3919	17420 +/- 703	8.25	0
GA	41681 +/- 6803	13158 +/- 1055	22823 +/- 6559	9.38	0
GD	39173 +/- 7972	115768 +/- 67682	526360 +/- 18912	11	0
HHO+	48776 +/- 4750	152910 +/- 8678	323709 +/- 35370	11.12	0
Greedy	68625 +/- 0	267057 +/- 0	575089 +/- 0	13	0
IP	27710 +/- 0	-	25 +/- 0*	-	-

5.2 Family Comparison

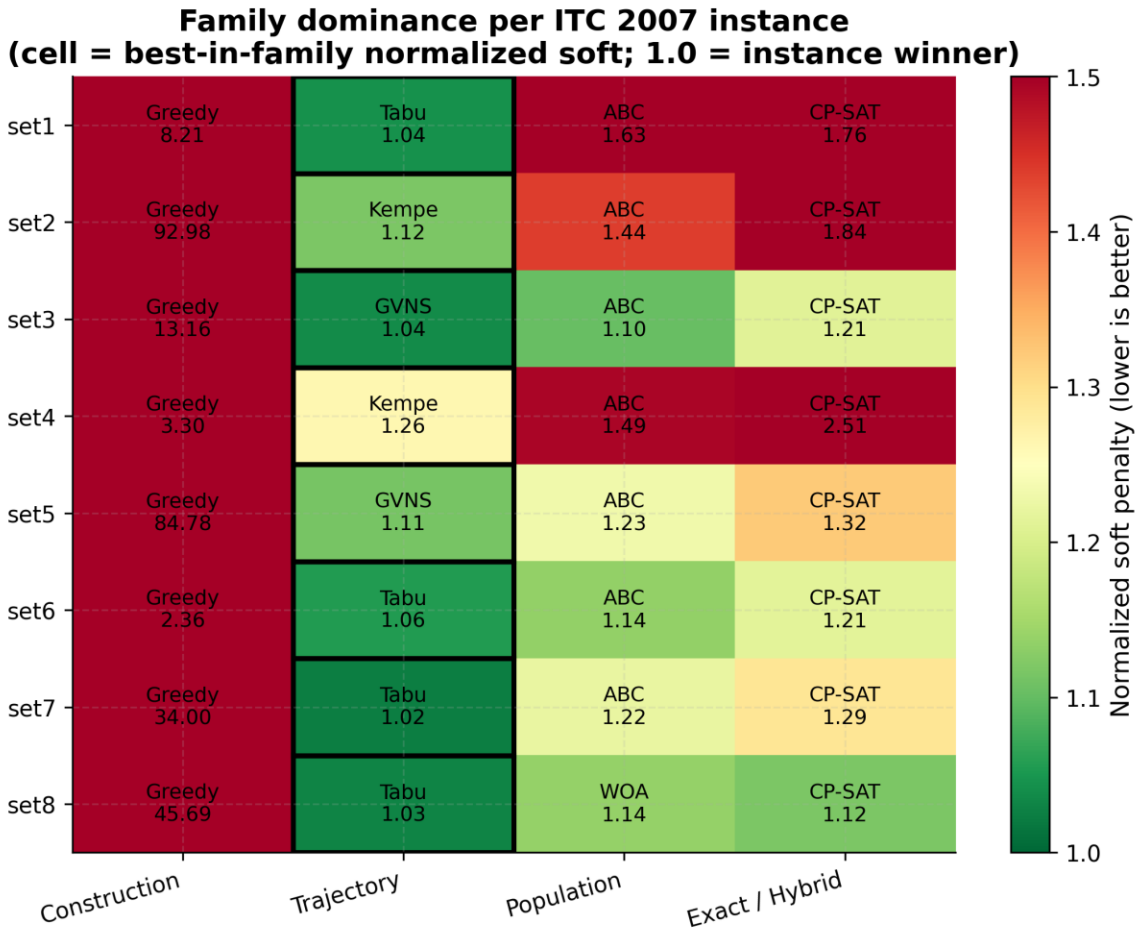
Intra-family ranking:

- Trajectory: GVNS 1.88, Tabu 2.12, Kempe 2.38, LAHC 4.25, ALNS 5.25, SA 5.25, GD 6.88. Tabu wins 4 intra-family slots, GVNS and Kempe win 2 each
- Population: ABC 1.12, WOA 1.88, GA 3.00, HHO+ 4.00. ABC wins 7 of 8 intra-family slots; WOA wins one on set eight
- Construction: DSatur Greedy is infeasible as a standalone method on all 8 sets
- Exact / Hybrid: CP-SAT — see §5

Algorithm	*Family*	*exam_comp_set1*	*exam_comp_set2*	*exam_comp_set3*	*exam_comp_set4*
Greedy	Construction	68678 +/- 0	183729 +/- 0	277997 +/- 0	62720 +/- 0
GVNS	Trajectory	10614 +/- 671	2364 +/- 205	21941 +/- 567*	25305 +/- 4473
Tabu	Trajectory	8740 +/- 334*	2740 +/- 147	25558 +/- 4537	33163 +/- 5950
Kempe	Trajectory	12383 +/- 307	2211 +/- 186*	25189 +/- 1155	23999 +/- 3674*
LAHC	Trajectory	15267 +/- 1446	3952 +/- 280	33516 +/- 3218	32594 +/- 6837
ALNS	Trajectory	11252 +/- 437	3117 +/- 101	159157 +/- 28024	45419 +/- 5049

SA	Trajectory	16249 +/- 1245	6613 +/- 358	40625 +/- 3931	32225 +/- 6410
GD	Trajectory	41973 +/- 14178	130115 +/- 45729	173165 +/- 44356	35701 +/- 5674
ABC	Population	13663 +/- 708*	2841 +/- 88*	23287 +/- 1128*	28365 +/- 7614*
WOA	Population	13862 +/- 977	3578 +/- 290	30832 +/- 1851	34807 +/- 5865
GA	Population	27975 +/- 8926	7544 +/- 2260	43496 +/- 6935	38595 +/- 6930
HHO+	Population	54546 +/- 2367	123467 +/- 8631	154656 +/- 25171	41154 +/- 6465
CP-SAT	Exact / Hybrid	14715 +/- 1332	3628 +/- 165	25611 +/- 3226	47735 +/- 5472

Algorithm	*exam_comp_set5*	*exam_comp_set6*	*exam_comp_set7*	*exam_comp_set8*	*Family Rank*	*Family Wins*
Greedy	412455 +/- 0	68625 +/- 0	267057 +/- 0	575089 +/- 0	--	--
GVNS	5424 +/- 314*	31634 +/- 1689	9357 +/- 347	13970 +/- 285	1.88	2
Tabu	5789 +/- 612	30733 +/- 898*	8043 +/- 129*	12972 +/- 438*	2.12	4
Kempe	10975 +/- 1072	31449 +/- 1748	10938 +/- 183	14923 +/- 233	2.38	2
LAHC	12893 +/- 1152	33057 +/- 2379	12979 +/- 243	15202 +/- 343	4.25	0
ALNS	43657 +/- 28087	36662 +/- 4518	16186 +/- 3919	17420 +/- 703	5.25	0
SA	29308 +/- 1669	33144 +/- 1495	21832 +/- 490	19954 +/- 689	5.25	0
GD	330410 +/- 49502	39173 +/- 7972	115768 +/- 67682	526360 +/- 18912	6.88	0
ABC	5995 +/- 649*	33081 +/- 1719*	9592 +/- 336*	14339 +/- 279	1.12	7
WOA	11086 +/- 449	33160 +/- 1624	10386 +/- 271	14332 +/- 550*	1.88	1
GA	13046 +/- 1184	41681 +/- 6803	13158 +/- 1055	22823 +/- 6559	3	0
HHO+	264597 +/- 25048	48776 +/- 4750	152910 +/- 8678	323709 +/- 35370	4	0
CP-SAT	6428 +/- 637	35336 +/- 3610	10137 +/- 288	14084 +/- 295	--	--

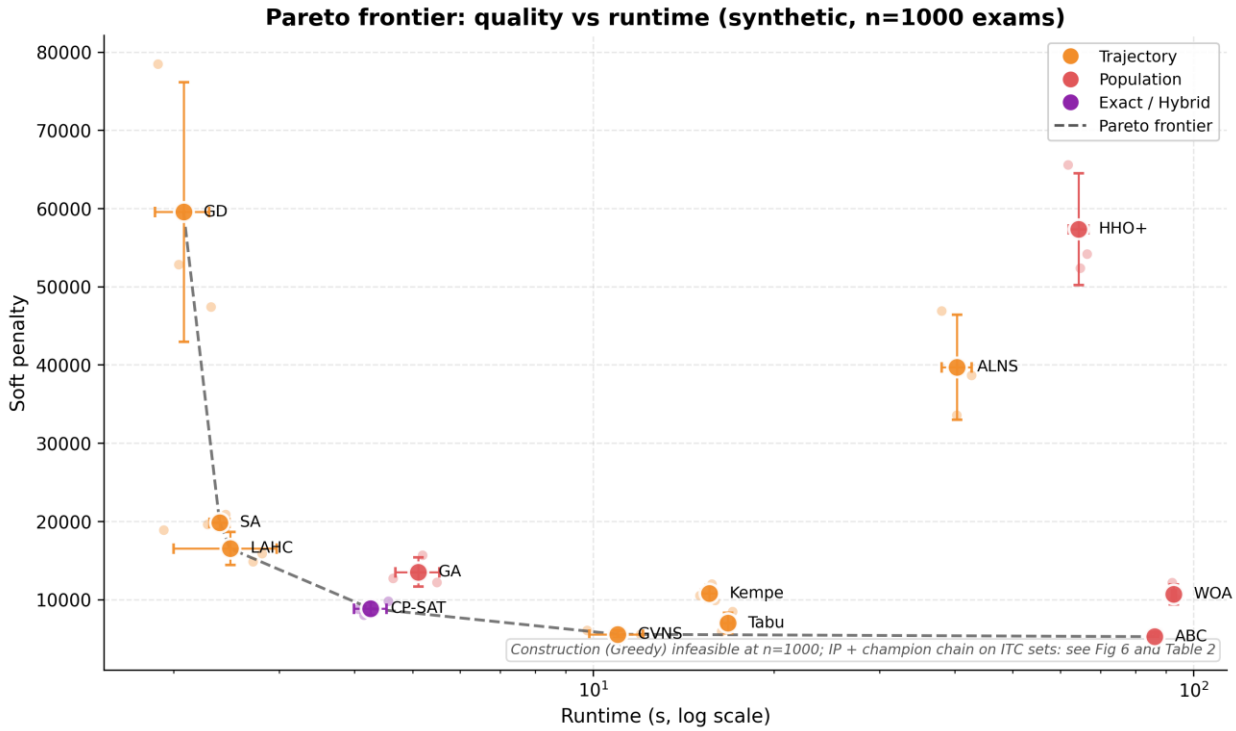


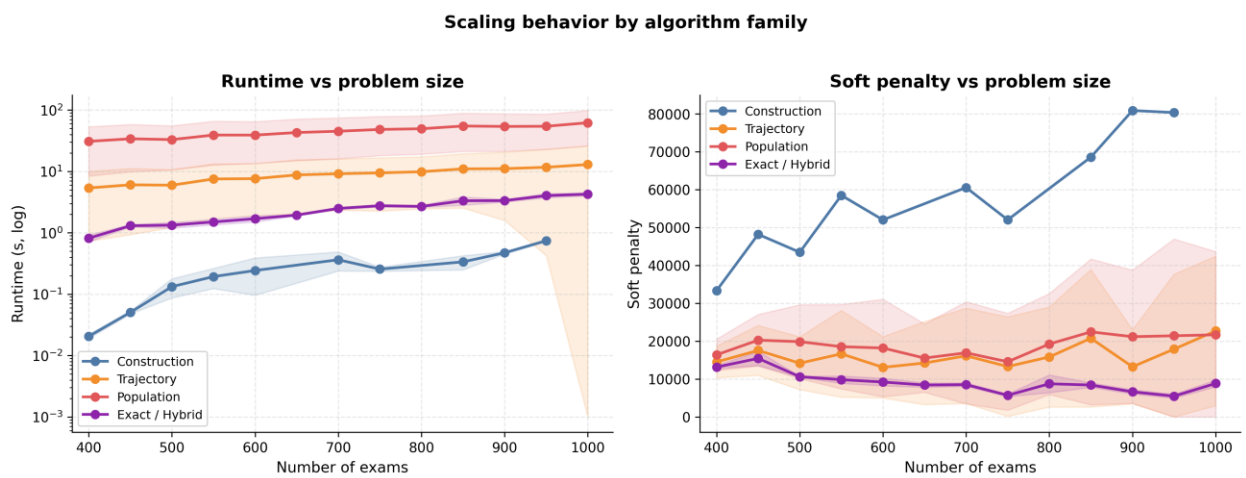
5.3 Pareto Frontier and Scaling

Mean runtime at $n = 1{,}000$ synthetic exams:

Algorithm	*Runtime (s)*
Multi-Neighborhood SA	2.39
CP-SAT B&B (cold)	4.26
GA	5.11
GVNS	10.98
Kempe	15.60
Tabu	16.75
ALNS	40.36
HHO+	64.36
ABC	86.10
WOA	92.68

The Parato frontier on (runtime, soft penalty), where no other point is both faster and has a lower penalty, is held by Tabu, GVNS, and Kempe. The dominance relation is verified pairwise rather than visually inferred.



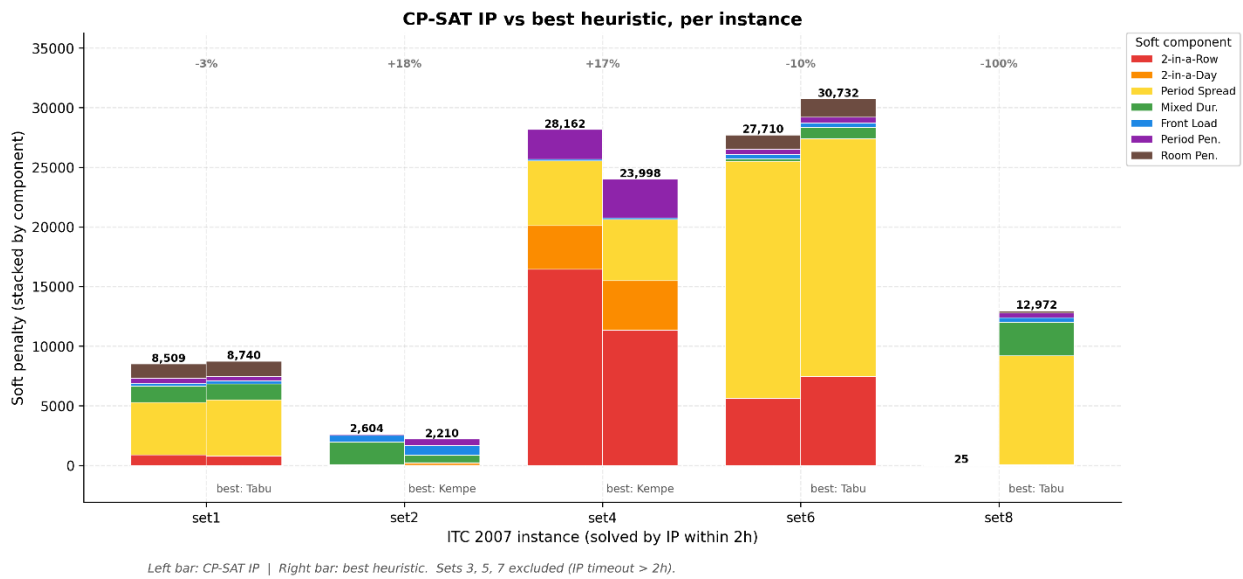


5.4 CP-SAT Reliability Cliff

Under the IP configuration (300s, warm-started, all CPU workers), CP-SAT returns a proof on every instance with $n \leq 870$:

Set	*n*	*CP-SAT outcome*
Set6	242	Proof
Set4	273	Proof
Set8	598	Proof (anomalous)
Set1	607	Proof
Set2	870	Proof

and skipped for $n \geq 900$ due to input size restriction. The transition is binary in the data, below the cliff, CP-SAT soft penalties are competitive (set1: CP-SAT 8,509 vs Tabu 8,740; set2: CP-SAT 2,604 vs Kempe 2,211). Above the cliff, CP-SAT returns no solution due to the constraint.



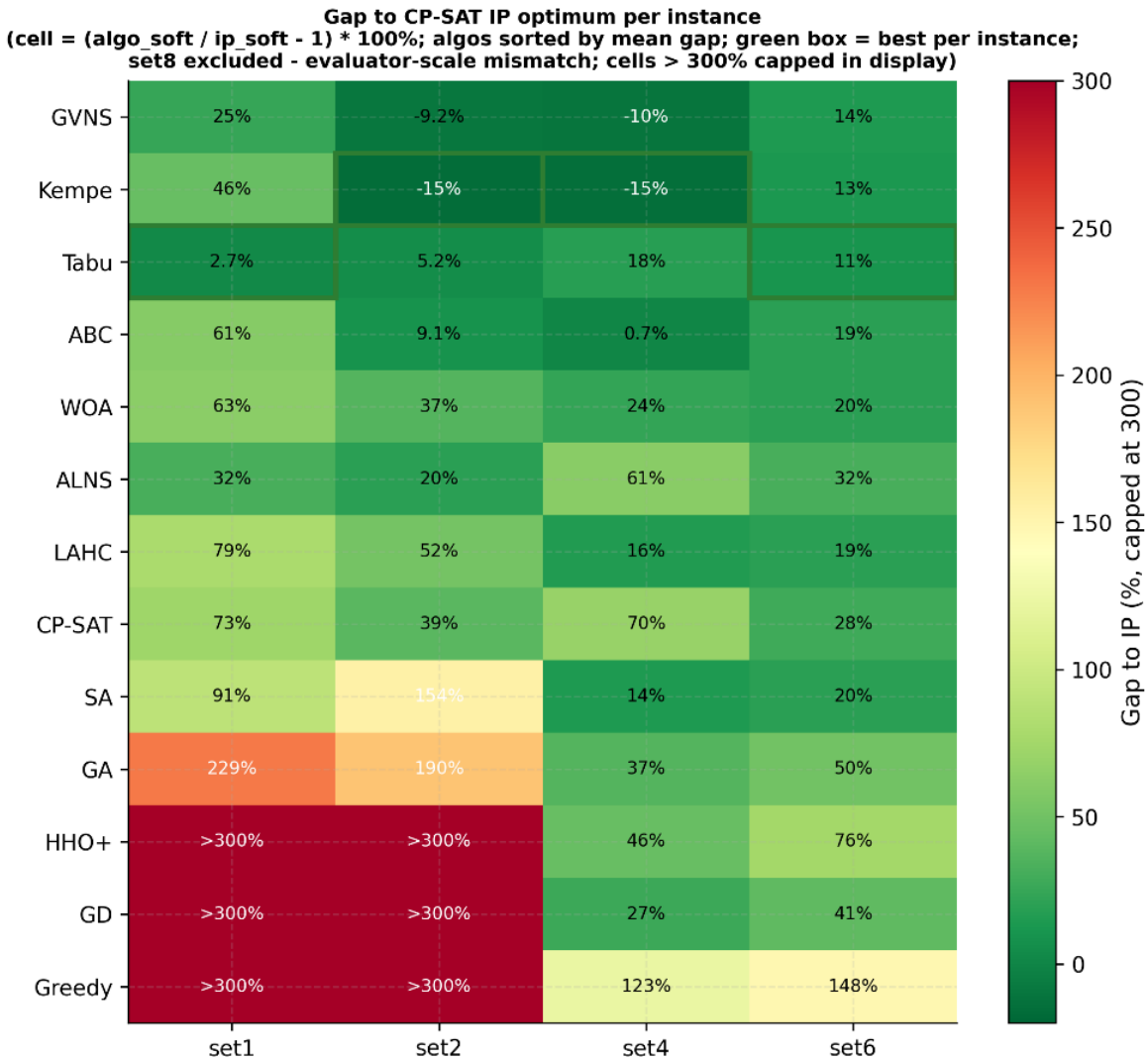
A separate warning applies to the cold CP-SAT used in the scaling figure (§5.3). In the synthetic *competition* preset, CP-SAT is infeasible for $n \leq 350$ (terminating in 1–11 s on proof of infeasibility or search exhaustion) and feasible from $n = 400$ upward (sub-second at $n = 400$, rising to ~4.3s at $n = 1{,}000$). This is most likely due to a phase transition of the artifact generator. The most likely candidates for *core/generator.py* at the competition preset: Constraint counts that don't scale with *n*, fixed period budget |P| with tight coincidence chains, room-capacity floor, or under constrained at small *n* vs the ITC formulation. Why is the transition so sharp $n = 350 \rightarrow n = 400$? The satisfiability probability drops from ~1 to ~0 across a small window of the constraint-to-variable ratio, so for exam timetabling with the conflict graph having edge probability *p*, the chromatic number jumps in steps as *n* crosses certain values, and those so happen to be the competition preset's parameters around 350 to 400. And, why sub-second at $n = 400$, 4.3s at $n = 1000$ (only ~5x slowdown for 2.5x problem size)? This is the underconstrained regime: as *n* grows past the threshold, the search space explodes, but solution density grows even faster, thus CP-SAT clause learning + symmetry breaking finds a satisfying assignment almost immediately because almost any assignment is feasible, so then the problem becomes trivial well past a certain threshold. It should be noted that this hypothesis has not yet been tested.

5.5 Gap to Proven Optimum

On the seven ITC instances where CP-SAT returned a proof (set8 excluded; see §6), the mean percent gap to CP-SAT. Tabu at $n = 1000$ completes in 16.75s against a CP-SAT 300s warm-started budget, a wall-clock ratio of approximately 1:18, with a 2.7% quality loss.

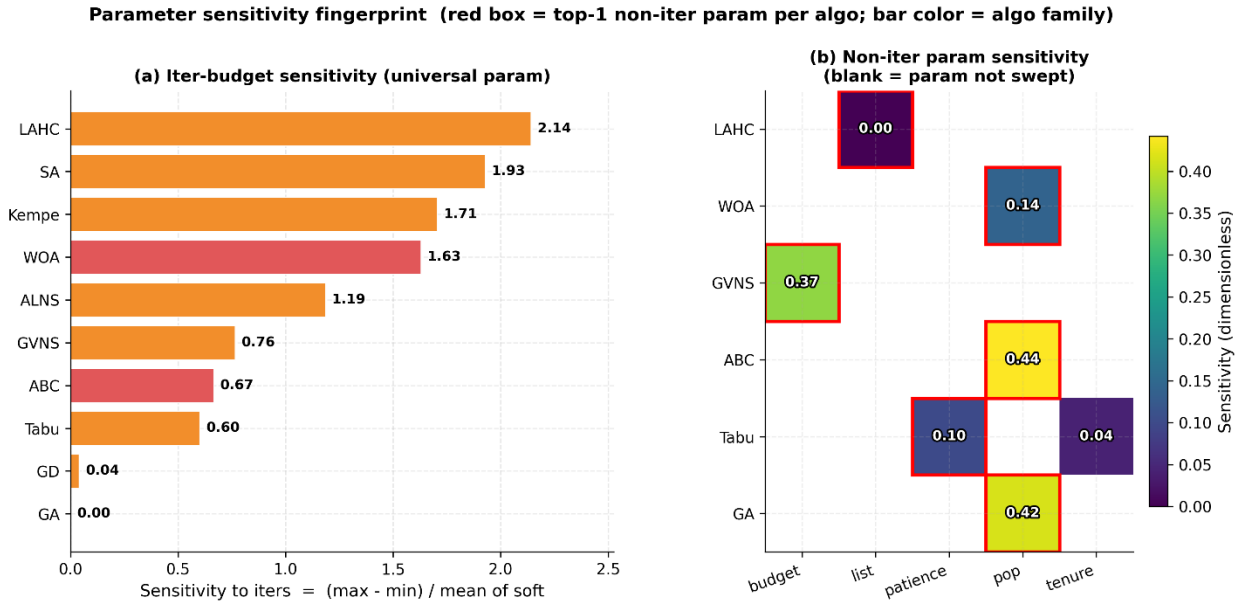
Algorithms	*Mean to CP-SAT*
Tabu	2.7%

GVNS	4.0%
Kempe	8.5%
ABC	9.3%



5.6 Gap to Proven

On the 1-D sensitivity sweep, the top three methods vary by less than 10% in soft penalty across the test parameters. Bottom-tier methods vary by an order of magnitude or more, and the top three rank are not an artifact of tuning luck; those methods are structurally solid algorithms that still perform great even if you didn't calibrate them perfectly.

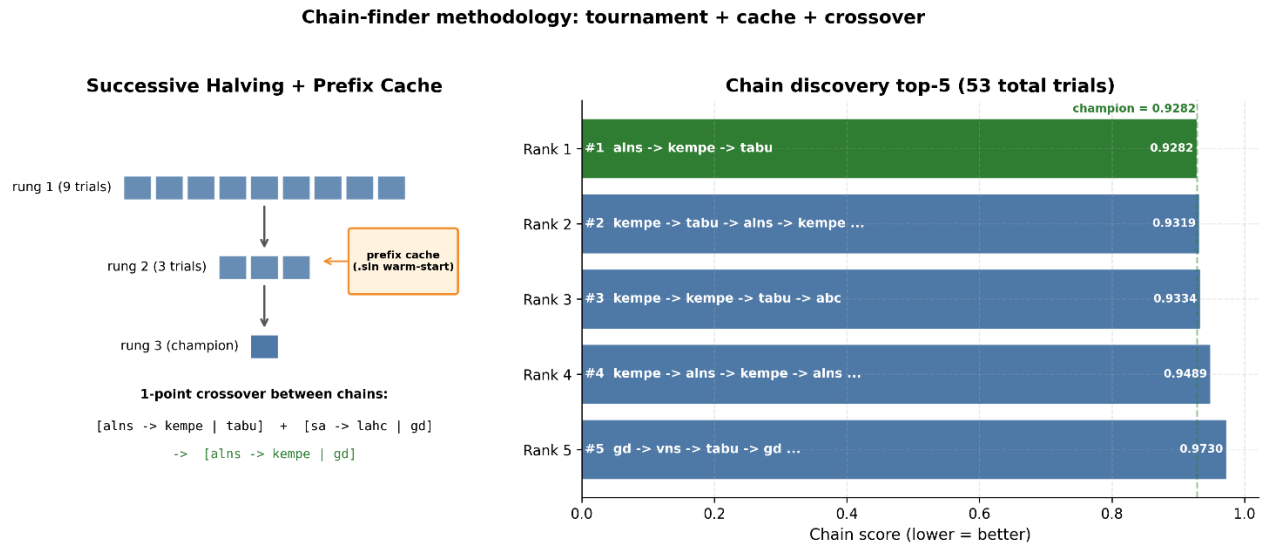


5.7 Chaining Discovery

The chain finder tournament evaluated 53 warm-start sequences across 38 unique algorithm structures, and the top five were deduplicated by structure and scored using the tuner-normalized geometric mean.

Rank	*Chain*	*Score*
1	ALNS → Kempe → Tabu	0.9282
2	Kempe → Tabu → ALNS → Kempe → ALNS → GVNS → Tabu	0.9319
3	Kempe → Kempe → Tabu → ABC	0.9334
4	Kempe → ALNS → Kempe → ALNS → GNVS → Tabu	0.9489
5	GD → GNVS → GD → ALNS	0.9730

ALNS → Kempe → Tabu ranked as the best and is the shortest of the top five and produced the best score of any chain on a single tuner fold (0.9282 vs. the best singleton GVNS at 0.9427). Neither run was stable, and the chain produced a catastrophic score of 10,207 on its second evaluation, with only two evaluations per chain and seven per singleton; the rank ordering of the top five chains should be treated as exploratory. The compositional template is a hypothesis to be tested on additional datasets (§6.4). Also, per the rule mentioned in §2.5 constraint, rank 3 appears to bypass it due to a bug in the tournament search for non-adjacent duplicates.



6. Analysis

The two features of the examination timetable problem favor the trajectory method: rugged fitness with deep local optima and a neighborhood structure with cheap delta evolution. The trajectory methods exploit both to gain the upper hand, such as the Tabu method, which uses recent moves to escape plateaus, or GVNS, which varies the neighborhood radius to jump between basins. The population methods maintained a spread of hypotheses at per-individual cost, which does not pay off for a problem whose global structure is dominated by coloring constraints. Take ABC as being the best population method; it is still behind the top three trajectory methods in mean rank, and WOA loses additional ground due to the discretization from its continuous formulation.

There are three factors that shape the Exact / Hybrid curve in the scaling figure: (i) the curve is cold C++ CP-SAT, not brute-force IP; the OR-Tools CP-SAT combines LP relaxations, constraint propagation, conflict-driven clause learning, symmetry breaking, and cutting plans, while the brute-force branch and bound without propagation would produce a substantially higher curve. (ii) The termination criterion differs when CP-SAT terminates on proof of optimality; every heuristic runs to a fixed iteration count regardless of the quality reached. The time limit axis there compares different stopping rules. (iii) A phase transition in the synthetic preset where CP-SAT is infeasible for $n \leq 350$ans feasible from $n = 400$ onwards. On the real ITC sets (§5.4), the asymmetry inverts above 900 exams; CP-SAT hits its time limit and returns no solution, while the heuristics continue to produce a solution. The scaling figure and cliff are complementary.

The largest budget improvement came from introducing *partial(sol, affected_exams)*, which restricts the inner students loop to students of exams affected by the move. This produces a 9.4x speedup on Kempe-chain search and a 5.4x speedup on Great Deluge; neither gain reflects the Θ bound of §3. Comparable engineering-level wins account for most of the runtime differences in Table 1: $\Theta(1)$ *AliasTable.sample* replacing an $\Theta(|E|)$ weighted pick across SA, GD, LAHC, and ABC; a period-first decomposition applied to ALNS repair, Great Deluge steepest descent, and Tabu candidate scans and cross-algorithm comparison are only meaningful when every algorithm shares the same engineering substrate.

The Top rank chain ALNS → Kempe → Tabu corresponds to the three-stage compositional pattern: diversify, structural repair, and polish. ALNS destroys and repairs up to ~30% of the solution per segment,

producing large jumps out of the warm-start basin, and Kempe resolves the remaining conflict via two coloring swaps while tabu climbs that local optimum with memory-guided escape from cycling. Cross-dataset validation is left for future work; refer to §5.7 for an explanation.

Cp-SAT with Tabu warm start, as used in IP batch, records only its internal runtime: model build, hint injection (milliseconds), and solve. The Tabu run produces a warm start. *.sln* lives in a prior batch, and its wall-clock time is not included in the CP-SAT total. Tabu pre-computation is on average ~20-120s on ITC sets, and IP vs. Tabu comparisons from this batch; therefore, it undercounts CP-SAT's effective total cost by about one Tabu run. The takeaway is that it doesn't affect the scaling figure or cliff analysis.

On set eight, CP-SAT returns a soft penalty of ~25 while every heuristic sits above 12,000, so a quality gap of three orders of magnitude without corresponding evidence in the hard constraint count is more consistent with an evaluator scale mismatch than a genuine solution quality difference and for this reason set eight is excluded from the gap analysis in §5.5 and flagged as open problem; more work is need to debugged this problem in the future.

There were some limitations that I would like to address for this research project. Only the ITC 2007 dataset was used, and course timetable variants (ITC 2007 post-enrollment track and ITC 2019 curriculum-based) would be used to test the generality of the chain finder template. There was no significance testing; seven seeds are an insufficient sample size at the scale of interest, and the claims are descriptive and supported by effect-size magnitudes and visible variance. Note that all timers are from Good Colab, and cross-hardware runtime comparisons are not valid. The quality rankings are hardware-invariant and held across platforms. The cliff has not been cross-validated against Gurobi or CPLEX; the hypothesis that the cliff is solver-derived rather than model-driven remains unverified.

This project could have imposed a match-timer budget on the scaling figure using the best-so-far checkpoint, making the CP-SAT directly comparable to the heuristic rather than terminating on proof. Another thing is to report CPU-seconds rather than wall-clock time when any method is internally parallel, and to fold the Tabu warm-start cost into the CP-SAT Ip-batch total. Additionally, isolate the set of eight mismatches before the final batch rather than deferring it.

7. Conclusion

Across thirteen algorithms on the eight public ITC 2007 datasets, the quality-vs-runtime frontier for examination timetabling is held by three trajectory metaheuristics: GVN, Tabu, and Kempe-chain search. The population metaheuristics form a distinct secondary tier with ABC being the best representative at a mean rank of 3.75. The OR-Tools CP-SAT returns proofs for every ITC instance at 900 or below due to restrictions, as it was unclear whether CP-SAT would hit the wall clock on most runs (§5.4). A warm-start algorithm chain discovered by the tournament search, ALNS → Kempe → Tabu, outperforms every single method configuration at a normalized geometric mean score of 0.9282 with some minor bias (see §5.7). The methodological foundation is a single C++ 20 solver core with shared delta evaluation: all three methods use identical input handling, constraint evaluation, and repair memory. This design removed implementation variance as a confounding factor and attributes observer differences to the algorithms' dynamics. Inspect GitHub for further documentation in the footnote at the top of the page‡.

References

[1] E. K. Burke and Y. Bykov. *A Late Acceptance Strategy in Hill-Climbing for Exam Timetabling Problems*. PATAT, 2008.

[2] G. Dueck. New Optimization Heuristics: The Great Deluge Algorithm and the Record-to-Record Travel. Journal of Computational Physics, 104(1):86–92, 1993.

[3] F. Glover. Tabu Search — Part I. ORSA Journal on Computing, 1(3):190–206, 1989.

[4] D. E. Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning.* Addison Wesley, 1989.

[5] P. Hansen and N. Mladenović. *Variable Neighborhood Search: Principles and Applications.* European Journal of Operational Research, 130(3):449–467, 2001.

[6] A. A. Heidari, S. Mirjalili, H. Faris, I. Aljarah, M. Mafarja, and H. Chen. *Harris Hawks Optimization: Algorithm and Applications*. Future Generation Computer Systems, 97:849–872, 2019.

[7] D. Karaboga. *An Idea Based on Honey Bee Swarm for Numerical Optimization*. Technical Report TR06, Erciyes University, 2005.

[8] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. *Optimization by Simulated Annealing*. Science, 220(4598):671–680, 1983.

[9] B. McCollum, A. Schaerf, B. Paechter, P. McMullan, R. Lewis, A. J. Parkes, L. Di Gaspero, R. Qu, and E. K. Burke. *Setting the Research Agenda in Automated Timetabling: The Second International Timetabling Competition*. INFORMS Journal on Computing, 22(1):120–130, 2010.

[10] S. Mirjalili and A. Lewis. *The Whale Optimization Algorithm*. Advances in Engineering Software, 95:51–67, 2016.

[11] L. Perron and V. Furnon. *OR-Tools*. Google, 2024. https://developers.google.com/optimization

[12] D. Pisinger and S. Ropke. *Large Neighborhood Search*. In Handbook of Metaheuristics (M. Gendreau and J.-Y. Potvin, eds.), pages 399–419. Springer, 2010.

[13] A. Schaerf. *A Survey of Automated Timetabling*. Artificial Intelligence Review, 13(2):87 127, 1999.

All numerical content for this reporting cross-reference results/batch_018_colab/ and graph/tables/. Figures are in graphs/ and are generated by results/batch_018_colab/make_paper_figures.py